

Full Research Plan

Deep Learning on Tabular Data

Supply Chain Demand Forecasting · M5 Walmart · 5 Models · 3 Experiments

Research Question	Do different types of temporal distribution shift differentially affect gradient-boosted trees vs. tabular deep learning models?
Novel Contribution	First study to decompose temporal shift into cold-start and temporal conditions on M5 Walmart and systematically evaluate all major tabular DL models.
Target Venues	IEEE Access (primary) · NeurIPS 2026 Tabular Workshop (secondary)

Contents

01 The Research Problem

What is tabular prediction

Why neural networks still lose

The core research gap

02 Research Question & Hypothesis

03 The Core Debate — Timeline

04 Dataset: M5 Walmart Competition

File structure

Scale and hierarchy

What the data looks like

05 Feature Engineering Strategy

Lag features

Rolling statistics

Calendar features

Price features

Categorical encoding

06 Experimental Design

E1 — Random split (baseline)

E2 — Temporal split (main)

E3 — Cold-start subset

07 The Five Models

XGBoost

LightGBM

TabPFN v2

TabM

TabR

08 Compute Strategy

09 Evaluation Framework

WAPE metric

Delta metric

Shift sensitivity

Statistical significance

10 Analysis Plan

11 Paper Writing Plan

12 Common Pitfalls

13 Glossary

3

3

3

4

4

5

6

6

6

7

7

7

8

8

8

8

8

9

9

9

10

11

11

12

13

14

15

16

17

17

17

18

18

18

20

22

23

01

The Research Problem

What tabular prediction is and why it matters

What Is Tabular Prediction?

Tabular data is any dataset organized into rows and columns — a spreadsheet, a SQL database table, a CSV file. Each row is one sample (a product sold on a given day at a given store), and each column is one feature (the day of the week, the selling price, the number of units sold). This is the dominant data format in enterprise machine learning. Retail demand forecasting, financial fraud detection, clinical risk scoring, and supply chain optimization all run on tabular data. It is not a niche format — it is the majority of real-world deployed ML systems.

The supply chain forecasting problem fits perfectly into this frame. Given a table of historical sales records for a product at a specific store, enriched with price information, calendar data, and promotional flags, the task is to predict future sales. This is a standard regression problem on tabular data.

Why Neural Networks Still Lose on Tabular Data

Since 2012, deep learning has replaced prior methods in image recognition, speech processing, and natural language understanding. The expectation was that neural networks would eventually dominate tabular data too. That expectation has not materialized. Gradient-boosted tree models — XGBoost and LightGBM — still outperform purpose-built neural architectures on most tabular datasets.

Structural Property	Why Trees Win
Irregular feature scales	Neural networks are sensitive to differences in feature magnitude across columns. Trees split on rank orders and are completely immune to this.
High-cardinality categoricals	Product IDs with millions of unique values. Trees handle them via native split rules. Neural networks require learned embeddings and far more data to generalize.
Sparse informative features	Often only 3 to 5 of 100 columns carry real signal. Trees find them via greedy splitting. Neural networks tend to overfit to noise in uninformative columns.
Small to medium scale	Most real production tables are under 1 million rows. Neural networks are sample-inefficient relative to trees at this scale.
Temporal non-stationarity	Data distribution changes over time. This is the core problem in supply chain forecasting and the gap this research studies.

The Core Research Gap

The landmark NeurIPS 2022 paper by Grinsztajn et al. proved trees outperform neural networks across 45 datasets. TabReD (ICLR 2025) reinforced this under temporal distribution shift on 8 industry datasets. TabPFN v2 (Nature 2025) showed neural methods win on small static data. Both findings are correct — in different regimes. The boundary between those regimes is unknown.

RESEARCH GAP

No paper has decomposed temporal shift into distinct types (cold-start, promotional, trend, macro shock) or studied which model fails at which type. This is the gap this project fills.

02

Research Question & Hypothesis

The precise claim this study makes

RESEARCH QUESTION

Under temporal distribution shift in retail demand forecasting, do different types of shift differentially affect gradient-boosted tree models versus tabular deep learning models, and does knowing the shift type enable better model selection?

Three Sub-Questions

Q1 — Replication	Does the TabReD finding hold on M5 Walmart? Do trees still beat neural models when a proper time-based split is used instead of a random split?
Q2 — Decomposition	Under cold-start conditions (products with minimal history), does TabPFN v2's in-context learning advantage over trees emerge? At what training history size (the crossover N) do trees start winning?
Q3 — Practical Guidance	Can a simple routing rule — assign cold-start products to TabPFN, all others to LightGBM — outperform either model used alone across the full test set?

Hypotheses

H1	Trees outperform neural models on the full temporal split (E2), consistent with TabReD
H2	TabPFN v2 outperforms trees on cold-start products (E3) with fewer than N training rows
H3	The crossover N exists between 20 and 200 training rows
H4	A routing strategy combining both model classes improves WAPE over either alone

03

The Core Debate — Timeline

How the research field arrived at this open question

Year	Venue	Paper	What It Proved	What It Left Open
2021	NeurIPS	FT-Transformer & SAINT	First serious transformer architectures for tabular data. Competitive on academic benchmarks.	Benchmarks used random splits. No production-scale or temporal testing.
2022	NeurIPS	Grinsztajn et al. — Trees Beat Nets	Across 45 datasets with proper statistical testing, trees consistently outperform neural nets. Key reasons: robustness to irregular features and uninformative columns.	Random splits only. No temporal structure. No cold-start analysis.
2022	NeurIPS	TabPFN v1	Prior-data Fitted Network using synthetic pretraining. Fast inference with no gradient updates. Competitive on very small datasets (under 1,000 rows).	Only works on very small datasets. No temporal evaluation.
2024	ICLR	TabR	Feed-forward network plus k-NN retrieval. Matches trees on standard academic benchmarks.	Standard random splits. No temporal or cold-start evaluation.
2025	ICLR	TabReD	With time-based splits, simpler MLPs and tree models outperform attention architectures. Academic benchmarks are misleading for production scenarios.	Does not decompose shift types. Does not test TabPFN v2. No routing strategy.
2025	ICLR	TabM	Parameter-efficient MLP ensembling beats FT-Transformer, SAINT, and TabR. Scales to 13M rows.	Not fully evaluated on TabReD-style temporal splits. No supply chain dataset.
2025	Nature	TabPFN v2	Beats 4-hour-tuned tree ensembles in 2.8 seconds on small data. In-context tabular learning is viable. Published in Nature.	Never tested on large-scale temporal datasets like M5. Cold-start unstudied.
2025	ICML	TabICL & TabPFN Unleashed	Extend in-context learning to 60,000+ rows. Beat XGBoost/LightGBM on 80% of TabArena.	Random or standard splits only. No temporal shift evaluation.
Open	—	YOUR RESEARCH	—	Which shift types cause which models to fail? The crossover conditions are unknown.

04

Dataset: M5 Walmart Competition

Full specification of the data you will work with

Why M5?

The M5 Forecasting Competition dataset, released by Walmart in partnership with the Makridakis Open Forecasting Center, is the canonical benchmark for retail demand forecasting. It is freely available on Kaggle, has been used in hundreds of published papers, and contains exactly the kind of temporal distribution shift this study targets. The M5 dataset is already available pre-loaded on Kaggle — no manual download is needed when running notebooks there.

Dataset Specification

Property	Value	Notes
Total rows	42 million	After melting from wide to long format
Unique products	30,490 SKUs	Across 10 product categories
Stores	10 Walmart stores	3 US states: California, Texas, Wisconsin
Time range	5 years (1,913 days)	January 2011 to June 2016
Training days	Days 1 to 1,521	Years 1 through 4
Test days	Days 1,522 to 1,913	Year 5 — never touch during training
Target variable	Unit sales per day	Integer; can be zero
Available on	Kaggle (free)	Pre-loaded in Kaggle notebook environments

The Three Source Files

File	What It Contains	Key Columns
sales_train_evaluation.csv	Main file. Wide format: one row per product, one column per day. Contains all unit sales.	id, item_id, dept_id, cat_id, store_id, state_id, d_1 through d_1969
calendar.csv	One row per day. Maps day codes (d_1, d_2...) to real dates and event information.	date, wm_yr_wk, weekday, wday, month, year, event_name_1, event_type_1, snap_CA, snap_TX, snap_WI
sell_prices.csv	Weekly selling price for each product at each store. Joined on store_id + item_id + wm_yr_wk.	store_id, item_id, wm_yr_wk, sell_price

Dataset Hierarchy

M5 has a hierarchical structure: each sale record belongs to an item, a department, a category, a store, and a state. This hierarchy matters for evaluation — WAPE can be computed at any level. Your study reports item-level WAPE (the most granular) as the primary metric.

Level	Count	Example
Item x Store	30,490	FOODS_3_090 at store CA_1
Item	3,049	FOODS_3_090 (across all stores)

Level	Count	Example
Department	7	FOODS_3, HOBBIES_1, HOUSEHOLD_2
Category	3	FOODS, HOBBIES, HOUSEHOLD
Store	10	CA_1, CA_2, CA_3, CA_4, TX_1...
State	3	CA, TX, WI
National	1	All stores combined

05

Feature Engineering Strategy

What features to create, why they matter, and how to construct them

Feature engineering is the most critical step for tree-based models and equally important for the neural models in this study. All five models consume a flat feature table — one row per product per day — with the same feature set. The target variable in each row is the unit sales on that day.

IMPORTANT — Data Leakage Prevention

All lag features and rolling statistics must be computed using only past values. A lag-7 feature for day D must use sales from day D-7, not day D. Compute these strictly in chronological order. Any leakage from future sales will produce inflated results that cannot be reproduced on real future data.

Lag Features

Most important feature group for demand forecasting

Feature Name	Description	Why It Matters
lag_7	Sales 7 days ago	Captures same-weekday effect. A Tuesday in week N uses Tuesday in week N-1.
lag_14	Sales 14 days ago	Two-week seasonality. Important for products with bi-weekly purchase cycles.
lag_28	Sales 28 days ago	Four-week (monthly) cycle. Standard in all M5 baseline papers.
lag_35	Sales 35 days ago	Five-week lookback. Helps smooth out holiday anomalies.

Rolling Statistics

Captures trend and volatility of recent demand

Feature Name	Description	Why It Matters
roll_mean_7	Mean of last 7 days	Short-term trend. Responds quickly to demand spikes.
roll_mean_28	Mean of last 28 days	Medium-term baseline. Less sensitive to one-off events.
roll_std_28	Std dev of last 28 days	Volatility measure. High variance products harder to forecast.
roll_max_28	Max of last 28 days	Captures peak demand — useful for promotions.

Calendar Features

Structural temporal signals known in advance

Feature Name	Description	Why It Matters
wday	Day of week (0-6)	Monday through Sunday. Huge effect on retail sales.
month	Month (1-12)	Captures annual seasonality.
week	ISO week number	Useful for weekly promotional cycles.

Feature Name	Description	Why It Matters
is_weekend	Binary flag	Weekends have fundamentally different demand patterns.
event_type_1	Event type (encoded)	Holidays, sporting events, cultural days — strong demand spikes.
snap_CA/TX/WI	SNAP subsidy flag	Government food assistance payment dates. Major signal for FOODS.

Price Features

Captures pricing dynamics and elasticity

Feature Name	Description	Why It Matters
sell_price	Current week selling price	Direct price signal.
price_change_7d	Price vs 7 days ago	Price increase vs decrease has asymmetric effects.
price_rank_cat	Price rank within category	Whether this item is currently cheap or expensive.
price_momentum_28	Price trend over 28 days	Sustained price change indicates new pricing tier.

Categorical Encoding

All categorical columns (item_id, store_id, dept_id, cat_id, state_id) must be numerically encoded before passing to any model. Use label encoding — assign each unique string value an integer. Do not use one-hot encoding at this scale; it would produce 30,000+ binary columns. Tree models handle label-encoded categoricals natively. Neural models benefit from learned embeddings, but label encoding is a valid starting point for all five models in this study.

06

Experimental Design

The three experiments, how to construct them, and what each tests

This study runs three experiments. E1 is the academic baseline (random split). E2 is the production-realistic condition (temporal split). E3 is the novel contribution (cold-start subset). Every model is evaluated on all three experiments using the same feature set and the same evaluation metrics.

E1 — Random Split (Baseline)

Purpose

Replicate standard academic benchmark conditions. This is what most tabular DL papers report. It is known to be overly optimistic because future data leaks into training.

Property	Specification
Train set	80% of all rows, sampled randomly without regard to time
Test set	Remaining 20%, randomly sampled
Data leakage	Present by design — future sales appear in training set. This is the point.
Expected result	Best WAPE numbers for all models. Trees and neural methods both look good here.
Why include it	Needed as the denominator for the Delta metric. Without E1, you cannot compute shift degradation.

E2 — Temporal Split (Main Experiment)

Purpose

Replicate the TabReD (ICLR 2025) methodology on M5. This is the honest evaluation that reflects real production conditions. Models must predict the future without seeing any future data during training.

Property	Specification
Train set	All rows where d_int is less than or equal to 1,521 (days 1 through 1,521 = years 1-4)
Test set	All rows where d_int is greater than 1,521 (days 1,522 through 1,913 = year 5)
Data leakage	None — test period is strictly after training period
Expected result	Worse WAPE for all models vs E1. How much worse = Delta metric.
Why it matters	This is the setting that mirrors real deployment. TabReD proved trees win here.
Validation set	Use days 1,400 to 1,521 as a held-out validation window for hyperparameter tuning. Do not use the test set for tuning.

E3 — Cold-Start Subset (Novel Contribution)

Purpose

Isolate the cold-start regime — products with very little historical data — and test whether TabPFN v2's in-context learning advantage emerges when trees have insufficient signal to learn from. This is the primary novel finding.

Property	Specification
Definition	Any product (id) that has fewer than 30 non-zero sales days in the training period
Train set	The training rows for cold-start items only, using the same temporal split as E2
Test set	The year-5 rows for cold-start items only
Expected scale	Typically 2,000 to 5,000 items out of 30,490 total qualify as cold-start
Why 30 days	Threshold from demand forecasting literature. Trees need approximately 50+ rows to build reliable splits. Below this, in-context learning should dominate.
Key experiment	Vary the threshold from 10 to 500 rows and track where each model's WAPE crosses. This produces the crossover curve (Figure 2 of your paper).

07

The Five Models

Full specification of each model: what it is, how it works, what to tune

XGBoost

Gradient-Boosted Trees · CPU · Baseline

What It Is

XGBoost (Extreme Gradient Boosting) is an ensemble method that builds decision trees sequentially, with each new tree correcting the residual errors of the previous ones. It has been the dominant tabular ML algorithm since winning multiple Kaggle competitions in 2016. It is the first baseline to run in this study.

How It Works Conceptually

XGBoost fits a series of shallow decision trees where each tree learns to predict the gradient of the loss function from the current ensemble's predictions. Trees are added one at a time; each one focuses on the samples the current model gets wrong. The final prediction is the sum of all tree outputs multiplied by a learning rate. The algorithm uses a second-order Taylor expansion of the loss for more accurate gradient computation than standard gradient boosting.

Key Hyperparameters to Tune

Parameter	Recommended Range	What It Controls
n_estimators	500 to 1,500	Number of trees. Use early stopping to find optimal value.
learning_rate	0.01 to 0.10	Step size per tree. Lower = more trees needed, better generalization.
max_depth	5 to 9	Maximum depth of each tree. Deeper = more complex = higher variance.
subsample	0.7 to 0.9	Fraction of training rows used per tree. Prevents overfitting.
colsample_bytree	0.6 to 0.9	Fraction of features used per tree. Adds diversity.
min_child_weight	1 to 50	Minimum sum of instance weight in a leaf. Higher = smoother model.
tree_method	hist (fixed)	Use histogram-based algorithm. Mandatory for M5 scale.
objective	reg:absoluteerror	MAE objective. Aligns with WAPE evaluation metric.
early_stopping	50 rounds	Stop training if validation MAE does not improve for 50 rounds.

Strengths

- Immune to feature scaling
- Handles missing values natively
- Fast training on CPU
- Excellent out-of-box performance
- Stable and predictable behavior

Limitations

- Slower than LightGBM on large data
- No native temporal awareness
- Requires tuning to avoid overfitting
- Cannot do true in-context learning

LightGBM

Gradient-Boosted Trees · CPU · Primary Tree Baseline

What It Is

LightGBM (Light Gradient Boosting Machine) is Microsoft's implementation of gradient boosted trees, designed specifically for large-scale datasets. It grows trees leaf-wise rather than depth-wise (unlike XGBoost), making it significantly faster on large datasets while achieving similar or better accuracy. It is the primary tree baseline and the model TabPFN v2 must beat on the cold-start experiment.

Key Difference From XGBoost

XGBoost grows trees level by level (all leaves at the same depth are split before going deeper). LightGBM grows trees leaf by leaf, always splitting the leaf with the highest potential gain regardless of depth. This makes LightGBM faster and often produces better results on large tabular datasets, at the cost of higher overfitting risk on small data.

Parameter	Recommended Range	What It Controls
num_leaves	63 to 255	Maximum number of leaves. More leaves = more complex model.
learning_rate	0.01 to 0.10	Step size per tree. Use with n_estimators and early stopping.
min_child_samples	20 to 200	Minimum data in a leaf. Critical regularizer for M5 scale.
feature_fraction	0.6 to 0.9	Fraction of features selected per tree iteration.
bagging_fraction	0.7 to 0.9	Fraction of data used per iteration (set bagging_freq=1 too).
lambda_l1	0 to 1	L1 regularization. Helps on high-cardinality feature sets.
lambda_l2	0 to 10	L2 regularization. Primary regularizer to prevent overfitting.
objective	regression_l1	MAE objective (L1). Aligns with WAPE. More robust than MSE.
n_estimators	500 to 2,000	Use early stopping with 50-100 rounds patience.

TabPFN v2

Tabular Foundation Model · RTX 3050 · In-Context Learning

What It Is

TabPFN v2 (Tabular Prior-data Fitted Network, version 2) is a transformer model pretrained on millions of synthetic tabular datasets generated from a prior distribution over datasets. Published in Nature in January 2025, it represents the paradigm shift that established tabular deep learning as a mainstream research area. Crucially, TabPFN has no training loop at deployment time — it uses in-context learning to perform inference without any gradient updates.

How In-Context Learning Works Here

At inference time, you provide TabPFN with a context window: a subset of labeled training examples alongside the unlabeled test examples you want to predict. The model processes all of them in a single forward pass and produces predictions by attending over the context. This is fundamentally different from fitting a model — there is no optimization happening. The model has already learned, during pretraining on synthetic data, how to generalize from small labeled datasets. This is why it excels at cold-start: it can generalize from 10 training examples where a tree has almost no signal.

Critical Constraints

Constraint	Value	Implication for This Study
Context window size	~10,000 rows	You must subsample the training set. Use 8,000 rows as context.
Training required	None	No GPU training loop. Install, fit on sample, predict. Done.
Inference speed	Seconds	Much faster than training tree models.
Dataset size limit	~10,000	Only suited for E3 cold-start subsets natively. Full M5 needs chunking.
Variance	High	Run 5 random seeds with different context samples. Report mean and std.
Hardware	RTX 3050	Your 3050 is fully sufficient. This is the only model that runs locally.

Strengths	Limitations
• Zero training time • Excels on small data / cold-start • Strong out-of-box performance • Natural fit for few-shot regimes • Runs on RTX 3050	• Hard limit of ~10K context rows • Requires subsampling for full M5 • Higher variance — needs multiple seeds • Not designed for large-N tabular

TabM

MLP Ensemble · Kaggle T4 · Parameter-Efficient

What It Is

TabM (from ICLR 2025, Yandex Research) achieves high tabular prediction accuracy through parameter-efficient ensembling of multiple MLP (multi-layer perceptron) predictions from a single shared backbone. It does not use attention or retrieval — it is essentially an ensemble of MLPs that share most of their parameters, making diverse predictions with minimal additional cost. It consistently outperforms FT-Transformer, SAINT, and TabR on both academic and real-world benchmarks.

Why It Is in This Study

TabM represents the strongest simple neural baseline — the result that tells you what a well-engineered MLP can achieve on tabular data when properly regularized and ensembled. It is the counterargument to 'you need attention for good tabular DL.' If TabM beats TabR in your results, that is itself a finding.

Parameter	Value	Notes
d_main	256 to 512	Hidden dimension of the MLP backbone.
n_heads	8 to 32	Number of ensemble members. More heads = more diversity but slower.
d_multiplier	2.0	Intermediate layer width multiplier. Keep at paper default.
dropout	0.0 to 0.2	Start at 0.0 as recommended in paper. Add if overfitting.
learning_rate	1e-3 to 3e-4	Use AdamW optimizer. Start at 1e-3.
weight_decay	1e-5 to 1e-4	L2 regularization via AdamW.
batch_size	2,048 to 8,192	Larger batches train faster on T4 GPU.
epochs	100 to 200	Use early stopping on validation MAE with 20 epoch patience.

TabR

Retrieval-Augmented Neural Net · Kaggle T4 · Slowest Model

What It Is

TabR (from ICLR 2024, Yandex Research) combines a feed-forward neural network with a differentiable k-NN retrieval mechanism. For each sample being predicted, TabR looks up the most similar training samples (nearest neighbors) and incorporates their labels into the prediction via a learned attention-like aggregation. This injects local inductive bias — the intuition that similar inputs should have similar outputs — directly into the neural network.

Why It Is Interesting

TabR is theoretically the most principled neural tabular architecture in this study. The k-NN retrieval is known to work well on tabular data; TabR makes it differentiable and end-to-end trainable. Its slowness is an inherent property of the retrieval mechanism — for each prediction, it must compare against the full training set. At M5 scale, you will need to subsample the training set to 500,000 rows maximum, or use the TabR-S (small) variant.

Parameter	Value	Notes
d_main	96 to 256	For TabR-S (fast), use 96. For full TabR, use 256.
context_size	96 to 256	Number of nearest neighbors retrieved per sample.
dropout	0.1 to 0.3	Higher dropout regularization needed vs TabM.
learning_rate	1e-3 to 5e-4	AdamW optimizer. Lower lr than TabM.
context_freeze	After 4 epochs	Context representations frozen early for speed. Described in paper.
Train subsample	500K rows	Mandatory for M5. Note in paper as a limitation.
batch_size	1,024	Retrieval overhead limits batch size vs TabM.
Expected runtime	8-10 hrs	On Kaggle T4. Start as an overnight job.

WARNING — TabR Runtime

TabR is significantly slower than all other models in this study due to the retrieval mechanism. If you are behind schedule, drop TabR and report it as future work. Your paper remains valid with 4 models.

08

Compute Strategy

Which model runs where, why, and estimated times

The RTX 3050 covers the majority of this study. Only TabM and TabR on the full M5 dataset require Kaggle. Kaggle's free T4 GPU (16GB VRAM, 30 hours per week) is better than the RTX 3050 for large batch neural training, and M5 is pre-loaded on Kaggle eliminating any data transfer time.

Model	Where to Run	GPU Needed?	Full M5 Time	E3 Subset Time
XGBoost	Local CPU	No	50-70 min	5-10 min
LightGBM	Local CPU	No	30-45 min	3-7 min
TabPFN v2	Local RTX 3050	Yes (4GB OK)	N/A (chunked)	10-15 min
TabM	Kaggle T4 (free)	Yes (16GB)	2-3 hrs	15-20 min
TabR	Kaggle T4 (free)	Yes (16GB)	8-10 hrs	30-40 min

Kaggle Setup Instructions

Create a Kaggle account	Go to kaggle.com . Registration is free. Verify your phone number to unlock GPU access.
Locate the M5 dataset	Search for 'M5 Forecasting Accuracy' on Kaggle. The dataset is pre-loaded and does not require downloading. Add it directly to your notebook from the Data panel.
Upload your pre-built features	After Day 1 local processing, upload your parquet files as a new Kaggle dataset called my-m5-features. This lets Kaggle notebooks load your processed data instantly without re-running the feature engineering pipeline.
Enable GPU	In a Kaggle notebook, go to Settings on the right sidebar. Under Accelerator, select GPU T4 x2. Each session gives you up to 12 hours of continuous GPU time. Your weekly quota is 30 hours — enough for all experiments.
Save outputs after every run	Write all results to <code>/kaggle/working/</code> as JSON or CSV immediately after each experiment finishes. Kaggle preserves output files even after sessions end, and they are downloadable from the Output tab of your notebook.
Session management	Kaggle sessions can time out after long periods of inactivity. For TabR overnight runs, enable Keep Session Alive in the notebook settings, or schedule the run to complete within a single active session.

09

Evaluation Framework

Every metric used, what it means, and how to interpret it

Primary Metric: WAPE

WAPE (Weighted Absolute Percentage Error) is the industry-standard evaluation metric for retail demand forecasting. It is the ratio of total absolute prediction error to total actual demand. It is scale-independent (a product selling 1 unit and one selling 10,000 contribute proportionally) and handles zero-sales days correctly unlike MAPE.

Property	WAPE
Formula	Sum of actual - predicted divided by Sum of actual
Range	0 to infinity. Lower is better. Perfect predictions give WAPE = 0.
Typical M5 range	0.5 to 0.8 at item level for well-tuned models
Zero handling	Graceful — denominator is sum of actuals, not per-item percentage
Scale invariance	Yes — a product selling 1 unit and 10,000 contribute proportionally

Key Metric: Delta (Shift Degradation)

Delta is this study's primary novel metric. It measures how much a model's performance degrades when moving from the academic random split (E1) to the production temporal split (E2 or E3). A positive Delta means the model suffers under temporal shift. A model with lower Delta is more robust to distribution shift.

Property	Delta Metric
Formula	$\Delta = (\text{WAPE on temporal split} / \text{WAPE on random split}) - 1$
Interpretation	Delta of +0.10 means 10% worse under temporal shift vs random split
Sign meaning	Positive: model degrades under shift. Zero: no effect. Negative: shift helps (rare).
Key comparison	If Δ_{E3} (cold-start) is lower for TabPFN than trees, that is your main finding
Example	LightGBM $E1=0.55$, $E2=0.63$ gives $\Delta_{E2} = (0.63/0.55)-1 = +0.145$ (+14.5% degradation)

Secondary Metric: Shift Sensitivity

Shift Sensitivity normalizes each model's Delta relative to the full temporal shift (E2 Delta). It answers: given that a model degrades X% on full temporal shift, how much more or less does it degrade specifically on cold-start? A Shift Sensitivity below 1.0 for TabPFN on E3 would mean it handles cold-start relatively better than temporal shift in general — your expected finding.

Statistical Significance

Report the Wilcoxon signed-rank test across the 30,490 items. This non-parametric test asks: across all items, does Model A produce lower absolute error than Model B more often than chance? This is more informative than a single aggregate WAPE number, especially when comparing models whose aggregate performance is close.

10

Analysis Plan

How to read, interpret, and present your results

The Master Results Table

Your central table is 5 models by 5 columns: E1 WAPE, E2 WAPE, E3 WAPE, Delta_E2, Delta_E3. Every number in this table is a result. The table goes in Section 5 of your paper and is the primary contribution of the empirical study.

Column	What It Shows	How to Compute
E1 WAPE	Academic baseline (random split)	Standard 80/20 random split WAPE
E2 WAPE	Production realistic (temporal)	Years 1-4 train, year 5 test WAPE
E3 WAPE	Cold-start regime	Products with < 30 training days
Delta E2	Temporal shift degradation	$(E2 / E1) - 1$, expressed as percentage
Delta E3	Cold-start degradation	$(E3 / E1) - 1$, expressed as percentage

Figure 1: Shift Sensitivity Bar Chart

A grouped bar chart with models on the x-axis and Delta percentage on the y-axis. Two bar groups per model: Delta_E2 (teal) and Delta_E3 (purple). This is the paper's main visual. If your hypothesis is correct, TabPFN v2 will show a substantially lower Delta_E3 bar than XGBoost and LightGBM, while having a higher or similar Delta_E2 bar. That visual contrast is the story of your paper.

Figure 2: Cold-Start Crossover Curve

A line chart with training history (number of rows, from 10 to 500) on the x-axis and WAPE on the y-axis. Two lines: TabPFN v2 and LightGBM. A vertical dashed line marks the crossover N — the point where LightGBM's line crosses below TabPFN's line. This is the paper's most actionable finding: TabPFN v2 for products with fewer than N historical rows, LightGBM otherwise.

Interpreting Your Findings

TabPFN v2 wins on E3, trees win on E2	This confirms both hypotheses. Your main finding: shift type determines model selection. The crossover N is the practical takeaway.
Trees win on both E2 and E3	A valid negative result. It means in-context learning does not overcome the structural disadvantage on cold-start at M5 scale. Report this honestly — it still extends TabReD.
TabPFN v2 wins on both E2 and E3	Surprising but publishable. Would contradict TabReD findings. Check your temporal split implementation for leakage before reporting.
TabM matches or beats TabR	Supporting finding: simple MLP ensembles are more practical than retrieval-augmented architectures on large-scale tabular data.
No model shows Delta below 0	Expected. All models should degrade under temporal shift. Zero or negative Delta would suggest data leakage — investigate immediately.

11

Paper Writing Plan

Section-by-section breakdown with what belongs in each part

Abstract150 wordsLast thing you write

Problem (1 sentence). Motivation (1 sentence). What you did (2 sentences). Key finding (1-2 sentences). Conclusion (1 sentence). Do not write this first.

1. Introduction1 to 1.5 pagesDay 7 morning first

Open with industry relevance: retail forecasting is a billion-dollar problem. Introduce the trees-vs-DL debate (cite Grinsztajn 2022 and TabReD 2025). State the open gap: no paper has decomposed shift types. State your 3-4 contributions as a numbered list. Close with paper structure overview.

2. Related Work1.5 to 2 pagesDay 7 late morning

Three subsections: (a) Tabular Deep Learning Methods — cite all 8 papers from Section 3 of this document. (b) Temporal Distribution Shift in ML — cite TabReD and dataset shift literature. (c) Demand Forecasting — cite M5 competition paper and TFT. End each subsection with an explicit gap statement.

3. Problem Formulation0.5 pagesDay 7 afternoon

Formally define temporal distribution shift. Define the E1, E2, E3 split conditions. Define WAPE and Delta mathematically. State the three hypotheses H1, H2, H3.

4. Dataset and Setup1 pageDay 7 afternoon

M5 description (scale, hierarchy, source). Temporal split methodology with exact day numbers. Feature engineering groups (lag, rolling, calendar, price, categorical). Model configurations and hyperparameter ranges. Compute environment.

5. Experiments and Results2 to 3 pagesDay 7 afternoon

One paragraph per experiment (E1, E2, E3). Master results table. Figure 1 (bar chart). Figure 2 (crossover curve). State each finding as a numbered finding (Finding 1, Finding 2, etc.).

6. Discussion1 pageDay 7 evening

Reconcile with TabPFN v2 (Nature 2025) and TabReD (ICLR 2025). Explain mechanistically WHY cold-start favors TabPFN. State practical implications: when to use each model. Limitations section: subsample size for TabR, single dataset, M5 range.

7. Conclusion0.5 pagesDay 7 evening

Restate contributions (4 bullets from Introduction). State main empirical finding in 2-3 sentences. Broader implications for tabular ML practitioners. Future work: additional shift types, more datasets, full routing strategy.

References25 to 35 citationsThroughout

All 8 papers from the core debate timeline. XGBoost and LightGBM original papers. M5 competition paper. Grinsztajn 2022. Statistical significance tests reference.

Abstract Template

Fill in your actual numbers in the brackets below after Day 6 analysis is complete.

Gradient-boosted trees consistently outperform neural methods on tabular data under temporal distribution shift, as demonstrated by TabReD (ICLR 2025) across eight industry datasets. However, no prior work has studied whether the type of temporal shift differentially affects model selection between tree-based and deep learning approaches. We address this gap by decomposing temporal shift into cold-start and temporal conditions using the M5 Walmart dataset (42 million rows, 30,490 products) and systematically evaluating five models: XGBoost, LightGBM, TabPFN v2, TabM, and TabR. We find that [MODEL] achieves [X]% lower WAPE degradation under cold-start conditions when training history is below [N] rows, while [MODEL] dominates with sufficient historical data (Delta_E2 = [X]% vs [Y]%). These results reveal a practical model selection principle: [YOUR RULE]. Code and data are available at [github.com/\[yourname\]/tabular-shift-study](#).

Target Venues

Venue	Type	Format	Difficulty	Deadline	Why Suitable
IEEE Access	Journal	8-12 pages	Low-Medium	Rolling	Empirical ML studies; no prestige barrier; 4-8 week review; undergraduate work accepted
NeurIPS 2026 Tabular Workshop	Workshop	4-8 pages	Medium	Sep 2026	Directly on-topic; workshop acceptance ~40-50%; high community visibility
ECML-PKDD 2026	Conference	12-15 pgs	Medium	Mar-Apr 2026	Applied ML flagship; replication-plus-extension papers accepted
Expert Systems w/ Applications	Journal	8-12 pages	Low-Medium	Rolling	Applied AI and ML; well-cited; tabular forecasting work regularly published

12

Common Pitfalls

Mistakes that invalidate results or waste time

CRITICAL ERROR — Using random splits for the main comparison

This is the entire point of TabReD. Random splits allow future data into training and produce optimistic results that cannot be reproduced in production. Always use time-based splits for E2. E1 random split is only for computing the Delta denominator.

CRITICAL ERROR — Tuning hyperparameters on the test set

Use days 1,400 to 1,521 as a validation window for all hyperparameter decisions. The test set (days 1,522 to 1,913) is touched once and only once: final evaluation. Any tuning against the test set invalidates the comparison.

WARNING — Forgetting to handle missing lag values at series start

Lag features for day 7 do not exist for days 1 through 6. These will be NaN. Fill them with the rolling mean of available history, or drop those rows from training. Never pass NaN to XGBoost or LightGBM without checking how they handle it.

WARNING — Comparing WAPE across hierarchy levels without normalizing

Item-level WAPE is always higher than store-level WAPE because aggregation smooths errors. All comparisons in this study must use the same hierarchy level (item level). Never compare an item-level WAPE from one model to a store-level WAPE from another.

WARNING — Running TabPFN v2 with a single seed

TabPFN's performance varies with the context sample chosen. With 8,000 rows sampled from 33 million, different random seeds select very different context windows. Run 5 seeds and report mean and standard deviation. A single seed result is not reliable.

WARNING — Using TabPFN on the full M5 without chunking

TabPFN has a hard context window limit. Attempting to pass 33 million training rows will fail immediately. Subsample to 8,000 rows as context. The model is designed for this regime — in-context learning does not benefit from context larger than ~10,000.

WARNING — Not saving intermediate results

TabM and TabR runs on Kaggle take hours. Kaggle sessions can time out. Save results to JSON after every experiment run. Do not wait until all experiments finish to save.

WARNING — Comparing metrics from different versions of the same model

If you run LightGBM for E1 with one hyperparameter configuration and E2 with another, the Delta metric is meaningless. Fix hyperparameters first (tuned on validation set), then run all three experiments with the same fixed configuration.

13

Glossary

Definitions of all technical terms used in this plan

Tabular Data	Data organized in rows (samples) and columns (features), like a spreadsheet or database table.
Temporal Distribution Shift	The statistical properties of the data change over time. A model trained on past data encounters different patterns in future data. This is the core challenge in supply chain forecasting.
Cold-Start	A product with very little or no historical sales data. Trees have insufficient signal to learn from. In-context learning methods like TabPFN are hypothesized to handle this better.
WAPE — Weighted Absolute Percentage Error	Sum of absolute errors divided by sum of actual values. The industry-standard metric for retail demand forecasting. Lower is better. Scale-independent.
Delta (Shift Degradation)	The percentage increase in WAPE when moving from random split (E1) to temporal split (E2 or E3). Quantifies how much a model suffers under distribution shift.
In-Context Learning	The ability to make predictions from a small number of labeled examples provided at inference time, without any gradient updates or training. TabPFN v2 uses this paradigm.
Lag Feature	The value of the target variable (sales) at a fixed number of days in the past. Lag-7 for day D is the sales on day D-7.
Rolling Statistic	A summary statistic (mean, standard deviation) computed over a sliding window of past values. Roll_mean_28 is the average sales over the past 28 days.
Gradient-Boosted Trees (GBT)	An ensemble of decision trees where each tree corrects the errors of the previous ones by fitting the gradient of the loss function. XGBoost and LightGBM are GBT implementations.
Prior-Data Fitted Network (PFN)	A neural network pretrained on synthetic datasets sampled from a prior distribution over datasets. At inference time it uses in-context learning rather than optimization.
TabPFN v2	Tabular Prior-data Fitted Network version 2. A tabular foundation model published in Nature (January 2025). Beats tuned tree ensembles on small data in 2.8 seconds.
TabM	Tabular Model with parameter-efficient ensembling (ICLR 2025). Multiple MLPs sharing a backbone, producing diverse predictions. Outperforms attention-based architectures.
TabR	Tabular Retrieval model (ICLR 2024). Combines a neural network with k-NN retrieval from the training set for each prediction. Slowest model in this study.
TabReD	Tabular Real Data benchmark (ICLR 2025). 8 industry datasets with time-based splits. The key predecessor paper showing trees still win in production settings.
SNAP Flags	Supplemental Nutrition Assistance Program payment flags in the M5 dataset. Indicate government food assistance payment dates by state. Strong signal for food category sales.
SKU	Stock-Keeping Unit. A unique identifier for a product at a specific location. M5 has 30,490 unique SKU identifiers.

Kaggle T4

NVIDIA T4 GPU available free on Kaggle notebooks. 16GB VRAM, 30 hours per week quota. Better than RTX 3050 for large-batch neural network training.

Early Stopping

A regularization technique where training stops when validation performance stops improving for a fixed number of rounds (patience). Prevents overfitting and saves compute time.

AdamW

Adam optimizer with weight decay (L2 regularization decoupled from gradient updates). Standard optimizer for TabM and TabR training.

Wilcoxon Signed-Rank Test

Non-parametric statistical test comparing two paired samples. Used here to test whether Model A produces lower absolute error than Model B across all items with statistical significance.

*The novelty is not a new model. The novelty is a new question asked with rigorous experimental design.
Run it cleanly, report it honestly, and it is publishable.*